Laboratorio 5.1: Primera Pantalla Declarativa

1. Contexto y Objetivos

En este laboratorio, los estudiantes realizarán la transición fundamental del modelo imperativo (XML) al modelo declarativo (Jetpack Compose). El objetivo principal es construir un **Validador de Credenciales en Tiempo Real**. A diferencia del enfoque tradicional donde el programador debe sincronizar manualmente cada cambio de la vista mediante findViewById(), en este laboratorio la UI se autogestionará basándose en el estado de los datos.

- Comprender la diferencia entre el "Cómo" (imperativo) y el "Qué" (declarativo).
- Implementar el manejo de estado básico usando remember y mutableStateOf.
- Construir interfaces reactivas que respondan a la entrada del usuario en tiempo real.

2. Requisitos Previos

- Haber completado la configuración del entorno según el **Laboratorio 1** (Android Studio Ladybug, API 36, JBR 21).
- Contar con la "Plantilla Maestra" creada en el primer laboratorio.

3. Descripción General

La aplicación consistirá en una pantalla única (LoginScreen) que contiene un campo de texto para el nombre de usuario. Debajo del campo, aparecerá un mensaje dinámico que cambiará de color y contenido según la longitud del texto ingresado, demostrando el poder de la recomposición en Compose.

4. Paso a Paso Detallado

Paso 1: Preparar el Proyecto (Clonación)

No iniciaremos un proyecto nuevo. Usaremos la consistencia técnica del Laboratorio 1.

1. En su explorador de archivos, copie la carpeta del **Laboratorio 1** y péguela con el nombre Laboratorio5_NombreApellido.
2. Borre las carpetas .gradle, .idea y build dentro de la nueva carpeta.
3. Abra la carpeta en Android Studio.
4. En settings.gradle.kts, cambie el rootProject.name a "Laboratorio5".
5. Haga clic en **Sync Project with Gradle Files** (el icono del elefante).

Paso 2: Localizar el Archivo de Trabajo

1. Navegue en el panel *Project* a: app/src/main/java/com/example/myapplication/MainActivity.kt.
2. Asegúrese de que el archivo tenga la estructura básica de una ComponentActivity con el soporte para Edge-to-Edge habilitado.

Paso 3: Definir el Estado y la Estructura

Reemplace el contenido del archivo con el siguiente bloque de código. Preste atención a los comentarios explicativos:

```
package com.example.myapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge() // Configuración Edge-to-Edge requerida
        setContent {
            MaterialTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    LoginScreen(modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }
}

@Composable
fun LoginScreen(modifier: Modifier = Modifier) {
    // 1. Definición del Estado: 'remember' hace que la variable sobreviva a la
recomposición
    var username by remember { mutableStateOf("") }

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        // 2. Construcción del DSL (Domain Specific Language) de UI
        OutlinedTextField(
            value = username,
            onValueChange = { username = it }, // Actualiza el estado al escribir
            label = { Text("Nombre de Usuario") },
            modifier = Modifier.fillMaxWidth()
        )
```

```
        Spacer(modifier = Modifier.height(8.dp))

        // 3. Lógica Declarativa: El sistema decide qué mostrar según el estado
        val isValid = username.length >= 3
        val message = if (isValid) "Nombre aceptado" else "Nombre muy corto (mín.
3)"
        val color = if (isValid) Color(0xFF2E7D32) else Color.Red // Verde vs Rojo

        Text(
            text = message,
            color = color,
            style = MaterialTheme.typography.bodyMedium
        )
    }
}
```

5. Resultados Esperados

Al ejecutar la aplicación en el emulador (Pixel 8 virtual API 36):

- Al abrir la app, verá un campo de texto vacío y el mensaje "Nombre muy corto" en rojo.
- A medida que escribe, el mensaje debe cambiar instantáneamente a "Nombre aceptado" en verde cuando alcance los 3 caracteres.
- No hubo necesidad de llamar a textView.setText(); la UI se redibujó sola al cambiar username.

6. Desafío Extra: Resolución

Reto: Añada un botón que se habilite solo si el nombre tiene 5 o más caracteres.
Solución: Elabore la solución

7. Entregables

1. Informe **PDF** con capturas de pantalla de:
    - Captura de pantalla mostrando la validación fallida (rojo) y otra con la validación exitosa (verde + botón habilitado).
    - Captura de pantalla que evidencie la Resolución del Reto.
    - Copia del código como Anexo (**MainActivity.kt**).
    - Respuestas a preguntas de haberlas.
    - Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.
2. **Enlace al repositorio de GitHub** con el commit final.

8. Criterios de Evaluación

No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe